

SETAPESU26/11_maze_chase

I am working on the existing Python/Pygame maze-chase game in this repository.

Implement ONLY Task 1 from the README:

"Multiple Enemies:

Add 2 more enemies starting at different corners. Each chases the player independently."

Requirements:

Show more

Worked for 51s

Implemented **Task 1 only** for the repository. The README confirms Task 1 is specifically to add two more independently BFS-driven enemies and store all three in `self.enemies`.

 GitHub

Changes made

Modified file: `game/game_engine.py` **only**

1. Replaced:

Python

Run

```
self.enemy = Enemy(ROWS-1, COLS-1)
```

with:

Python

Run

```
self.enemies = [  
    Enemy(ROWS-1, COLS-1), # bottom-right  
    Enemy(0, COLS-1),      # top-right
```

```
Enemy(ROWS-1, 0), # bottom-left
```

So there are exactly **3 Enemy instances**, each starting in a different corner.

2. Enemy updates now loop through all three:

Python

Run

```
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
```

This preserves the existing `Enemy.update()` implementation, which independently calls `bfs()` for each enemy.  [GitHub](#)

3. Collision detection now checks every enemy:

Python

Run

```
for enemy in self.enemies:
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
        break
```

4. Enemy drawing now loops through all three:

Python

Run

```
for enemy in self.enemies:
    enemy.draw(self.screen)
```

5. **Reset behavior is preserved** because `self.enemies` is recreated every time `reset()` runs.

6. Player movement, maze generation, EXIT behavior, win/lose overlays, and controls were left unchanged. The existing maze/BFS implementation was not modified.

 [GitHub](#)

7. **Tasks 2, 3, and 4 were not implemented.**

Verification

I checked the modified project for:

- Python syntax errors → **PASS**
- Exactly 3 `Enemy(...)` instances → **PASS**
- `self.enemies` list → **PASS**

- Independent `enemy.update()` loop → **PASS**
- Collision loop → **PASS**
- Drawing loop → **PASS**
- No remaining `self.enemy` references → **PASS**
- All three starting positions are distinct valid maze corners → **PASS**

One limitation: I could not launch the actual Pygame window in this environment because Pygame isn't installed here, so I verified the source syntactically and structurally rather than claiming a live graphical run.

Your modified project

[Download the Task 1 modified project ZIP](#)

The GitHub repository itself was not directly pushed/edited because this chat doesn't currently have write access to your GitHub repository. The ZIP contains the modified project with the exact Task 1 implementation.

The original repository structure and current single-enemy implementation were checked against [your GitHub repository](#).

I am working on the existing 11_maze_chase Python/Pygame repository. I will manually apply your changes to my existing local GitHub repository.

Implement ONLY Task 1 from the README:

MULTIPLE ENEMIES

- Add 2 additional enemies, giving 3 enemies total.
- Start the enemies at different valid maze corners/positions.

[Show more](#)

Worked for a couple of seconds

Yes. I checked the **current repository version directly** before giving you the code. Task 1 in the README explicitly requires three `Enemy` instances in `self.enemies`, with each independently calling the existing BFS through `Enemy.update()`.  [GitHub +1](#)

File you need to replace

You only need to replace:

```
game/game_engine.py
```

Do not change `game/entities.py`. Its existing `Enemy.update()` already calls `bfs()`

independently for each `Enemy` instance, so modifying it would be unnecessary.  [GitHub](#)

The three enemies will start at:

- **Bottom-right:** `(ROWS-1, COLS-1)` — the original enemy position
- **Top-right:** `(0, COLS-1)`
- **Bottom-left:** `(ROWS-1, 0)`

The player remains at `(0, 0)`.

Complete `game/game_engine.py`

game/game_engine.py — Task 1

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
```

```

# Task 1: Multiple Enemies
# Three enemies start at different valid maze corners.
self.enemies = [
    Enemy(ROWS - 1, COLS - 1), # bottom-right
    Enemy(0, COLS - 1),        # top-right
    Enemy(ROWS - 1, 0),        # bottom-left
]

self.exit_rect = pygame.Rect(
    (COLS // 2) * CELL + 5,
    (ROWS // 2) * CELL + 5,
    CELL - 10,
    CELL - 10
)

self.caught = False
self.won = False

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()

    return True

def update(self):
    if self.caught or self.won:
        return

    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)

    # Each enemy independently updates using its existing BFS pathfindi
    for enemy in self.enemies:
        enemy.update(self.walls, self.player, ROWS, COLS)

    # Check collision with every enemy.
    for enemy in self.enemies:
        if self.player.rect.colliderect(enemy.rect):
            self.caught = True

```

```
        break

    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))

    wc = (50, 40, 60)

    for r in range(ROWS):
        for c in range(COLS):
            x, y = c * CELL, r * CELL
            w = self.walls[r][c]

            if w[0]:
                pygame.draw.line(
                    self.screen,
                    wc,
                    (x, y),
                    (x + CELL, y),
                    3
                )

            if w[1]:
                pygame.draw.line(
                    self.screen,
                    wc,
                    (x, y + CELL),
                    (x + CELL, y + CELL),
                    3
                )

            if w[2]:
                pygame.draw.line(
                    self.screen,
                    wc,
                    (x + CELL, y),
                    (x + CELL, y + CELL),
                    3
                )

            if w[3]:
```

```

        pygame.draw.line(
            self.screen,
            wc,
            (x, y),
            (x, y + CELL),
            3
        )

pygame.draw.rect(
    self.screen,
    (80, 200, 80),
    self.exit_rect,
    border_radius=4
)

lbl = self.font.render("EXIT", True, (20, 80, 20))
self.screen.blit(
    lbl,
    (self.exit_rect.x + 2, self.exit_rect.y + 6)
)

self.player.draw(self.screen)

# Draw all three enemies.
for enemy in self.enemies:
    enemy.draw(self.screen)

hud = pygame.Rect(
    0,
    ROWS * CELL,
    WIDTH,
    50
)

pygame.draw.rect(
    self.screen,
    (30, 30, 50),
    hud
)

info = self.font.render(
    "Reach EXIT before the enemy catches you! R=Restart",
    True,

```

```

        (200, 200, 200)
    )

    self.screen.blit(
        info,
        (8, ROWS * CELL + 14)
    )

    if self.caught:
        self._overlay("CAUGHT!", (220, 60, 60))

    if self.won:
        self._overlay("ESCAPED!", (80, 220, 80))

    pygame.display.flip()

def _overlay(self, text, color):
    surf = pygame.Surface(
        (WIDTH, ROWS * CELL),
        pygame.SRCALPHA
    )

    surf.fill((0, 0, 0, 140))
    self.screen.blit(surf, (0, 0))

    msg = self.big_font.render(text, True, color)
    sub = self.font.render(
        "Press R to Restart",
        True,
        (200, 200, 200)
    )

    self.screen.blit(
        msg,
        (
            WIDTH // 2 - msg.get_width() // 2,
            ROWS * CELL // 2 - 30
        )
    )

    self.screen.blit(
        sub,
        (

```

```

        WIDTH // 2 - sub.get_width() // 2,
        ROWS * CELL // 2 + 20
    )
)

def run(self):
    running = True

    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)

pygame.quit()

```

What actually changed

Compared with your current repository's `game_engine.py`, there are only **three functional changes**. The original file currently creates one enemy with `self.enemy = Enemy(...)`, updates that enemy once, checks one collision, and draws one enemy.  [GitHub](#)

1. One enemy → three enemies

Python

Run

```

self.enemies = [
    Enemy(ROWS - 1, COLS - 1),
    Enemy(0, COLS - 1),
    Enemy(ROWS - 1, 0),
]

```

2. Update all enemies

Python

Run

```

for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)

```

This is important because **you do not need to modify BFS** or `Enemy.update()`. Each object has its own `r`, `c`, and `timer`, so every enemy independently calculates its own BFS path to the player's current cell.  [GitHub](#)

3. Draw and collide with all enemies

Python

Run

```
for enemy in self.enemies:
    enemy.draw(self.screen)
```

and:

Python

Run

```
for enemy in self.enemies:
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
        break
```

The existing **R** restart behavior is preserved because `reset()` now simply recreates all three enemies. The existing EXIT and win/lose behavior is also untouched.

 GitHub

Files you should have afterward

```
11_maze_chase/
|
├─ main.py           ← NO CHANGE
├─ requirements.txt  ← NO CHANGE
├─ README.md         ← NO CHANGE
|
└─ game/
    ├─ __init__.py    ← NO CHANGE
    ├─ game_engine.py ← REPLACE WITH CODE ABOVE
    ├─ maze.py        ← NO CHANGE
    └─ entities.py    ← NO CHANGE
```

This is deliberately limited to **Task 1**. Nothing from the speed ramp, power pellet, or score tasks has been added. The repository's README confirms those are Tasks 2–4.

 GitHub

Testing checklist

After replacing `game/game_engine.py`:

1. Open your existing repository in VS Code.
2. Run:

Bash

3. Confirm the game starts normally.
4. Confirm you can see **3 red enemies**.
5. Confirm they start at **three different corners**.
6. Move the player around the maze.
7. Confirm **all three enemies move toward the player independently**.
8. Let any enemy touch the player → `CAUGHT!`.
9. Press **R** → maze/game resets and **three enemies appear again**.
10. Reach the EXIT → `ESCAPED!`.
11. Confirm there is **no speed increase**, **no power pellet**, and **no score** — those belong to Tasks 2–4.
12. Optional syntax check:

Bash

```
python -m py_compile game/game_engine.py
```

Expected final state: exactly 3 independently moving BFS enemies, and nothing else from Tasks 2–4.

I am continuing from the EXISTING 11_maze_chase repository.

Task 1 is already implemented and working:

- There are exactly 3 enemies.
- They are stored in `self.enemies`.
- They independently use the existing BFS `Enemy.update()` behavior.

Now implement **ONLY** Task 2 from the README:

Show more

For Task 2, **only** `game/game_engine.py` **needs to change**. Your Task 1 implementation stays intact, including the three enemies and their existing BFS behavior.

The implementation below uses `pygame.time.get_ticks()` and the existing `Enemy.move_interval` property. It does **not** modify `entities.py` or the BFS system.

File to replace

Replace only:

```
game/game_engine.py
```

Do not replace or modify:

```
game/entities.py
game/maze.py
main.py
README.md
```

Complete `game/game_engine.py`

game/game_engine.py — Task 1 + Task 2

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)

        # Task 1: Multiple Enemies
        # Three enemies start at different valid maze corners.
        self.enemies = [
```

```

        Enemy(ROWS - 1, COLS - 1), # bottom-right
        Enemy(0, COLS - 1),        # top-right
        Enemy(ROWS - 1, 0),        # bottom-left
    ]

```

```

# Task 2: Speed Up Over Time

```

```

# All enemies start with a movement interval of 20.

```

```

for enemy in self.enemies:
    enemy.move_interval = 20

```

```

# Record when the current game started.

```

```

# This is reset whenever reset() is called, including when R is pre

```

```

self.speed_start_time = pygame.time.get_ticks()

```

```

self.exit_rect = pygame.Rect(
    (COLS // 2) * CELL + 5,
    (ROWS // 2) * CELL + 5,
    CELL - 10,
    CELL - 10
)

```

```

self.caught = False

```

```

self.won = False

```

```

def handle_events(self):

```

```

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

```

```

        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()

```

```

    return True

```

```

def update(self):

```

```

    if self.caught or self.won:
        return

```

```

    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)

```

```

# Task 2: Reduce the movement interval every 15 seconds.

```

```

#

```

```

# Starting interval: 20
# Every 15 seconds: -2
# Minimum interval: 5
#
# This produces:
# 0-14s    -> 20
# 15-29s   -> 18
# 30-44s   -> 16
# 45-59s   -> 14
# 60-74s   -> 12
# 75-89s   -> 10
# 90-104s  -> 8
# 105-119s -> 6
# 120s+    -> 5

elapsed_time = pygame.time.get_ticks() - self.speed_start_time
speed_steps = elapsed_time // 15000

current_interval = max(5, 20 - (speed_steps * 2))

# Apply the same interval to all three enemies.
for enemy in self.enemies:
    enemy.move_interval = current_interval

# Each enemy independently uses the existing Enemy.update()
# and its existing BFS pathfinding.
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)

# Check collision with every enemy.
for enemy in self.enemies:
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
        break

if self.player.rect.colliderect(self.exit_rect):
    self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))

    wc = (50, 40, 60)

```

```
for r in range(ROWS):
    for c in range(COLS):
        x, y = c * CELL, r * CELL
        w = self.walls[r][c]

        if w[0]:
            pygame.draw.line(
                self.screen,
                WC,
                (x, y),
                (x + CELL, y),
                3
            )

        if w[1]:
            pygame.draw.line(
                self.screen,
                WC,
                (x, y + CELL),
                (x + CELL, y + CELL),
                3
            )

        if w[2]:
            pygame.draw.line(
                self.screen,
                WC,
                (x + CELL, y),
                (x + CELL, y + CELL),
                3
            )

        if w[3]:
            pygame.draw.line(
                self.screen,
                WC,
                (x, y),
                (x, y + CELL),
                3
            )

pygame.draw.rect(
    self.screen,
```

```

        (80, 200, 80),
        self.exit_rect,
        border_radius=4
    )

    lbl = self.font.render("EXIT", True, (20, 80, 20))
    self.screen.blit(
        lbl,
        (self.exit_rect.x + 2, self.exit_rect.y + 6)
    )

    self.player.draw(self.screen)

    # Task 1: Draw all three enemies.
    for enemy in self.enemies:
        enemy.draw(self.screen)

    hud = pygame.Rect(
        0,
        ROWS * CELL,
        WIDTH,
        50
    )

    pygame.draw.rect(
        self.screen,
        (30, 30, 50),
        hud
    )

    # Task 2: Display the current enemy movement interval.
    info = self.font.render(
        f"Enemy interval: {self.enemies[0].move_interval}    R=Restart"
        True,
        (200, 200, 200)
    )

    self.screen.blit(
        info,
        (8, ROWS * CELL + 14)
    )

    if self.caught:

```

```

        self._overlay("CAUGHT!", (220, 60, 60))

    if self.won:
        self._overlay("ESCAPED!", (80, 220, 80))

    pygame.display.flip()

def _overlay(self, text, color):
    surf = pygame.Surface(
        (WIDTH, ROWS * CELL),
        pygame.SRCALPHA
    )

    surf.fill((0, 0, 0, 140))
    self.screen.blit(surf, (0, 0))

    msg = self.big_font.render(text, True, color)
    sub = self.font.render(
        "Press R to Restart",
        True,
        (200, 200, 200)
    )

    self.screen.blit(
        msg,
        (
            WIDTH // 2 - msg.get_width() // 2,
            ROWS * CELL // 2 - 30
        )
    )

    self.screen.blit(
        sub,
        (
            WIDTH // 2 - sub.get_width() // 2,
            ROWS * CELL // 2 + 20
        )
    )

def run(self):
    running = True

    while running:

```

```

        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)

pygame.quit()

```

What changed for Task 2

There are four relevant additions:

1. Timer starts when the game/reset starts

Python

Run

```
self.speed_start_time = pygame.time.get_ticks()
```

Because this is inside `reset()`, pressing **R** completely resets the speed progression.

2. All enemies explicitly start at 20

Python

Run

```
for enemy in self.enemies:
    enemy.move_interval = 20

```

3. Interval calculation

Python

Run

```

elapsed_time = pygame.time.get_ticks() - self.speed_start_time
speed_steps = elapsed_time // 15000
current_interval = max(5, 20 - (speed_steps * 2))

```

So the exact progression is:

Elapsed time	Enemy interval
0–14.999 s	20
15–29.999 s	18
30–44.999 s	16
45–59.999 s	14
60–74.999 s	12

Elapsed time	Enemy interval
75–89.999 s	10
90–104.999 s	8
105–119.999 s	6
120+ s	5

The `max(5, ...)` guarantees it can **never go below 5**.

4. The same interval is applied to all three enemies

Python

Run

```
for enemy in self.enemies:
    enemy.move_interval = current_interval
```

Their BFS logic remains untouched:

Python

Run

```
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
```

So this is still **three independent BFS enemies**, just with their movement interval changing simultaneously.

Testing

Run your existing project normally:

Bash

```
python main.py
```

Then check:

1. Immediately after starting

- HUD says:

```
Enemy interval: 20
```

- Three enemies are present.

2. Wait 15 seconds

- HUD should change to:

```
Enemy interval: 18
```

3. Continue waiting and verify:

```
20 → 18 → 16 → 14 → 12 → 10 → 8 → 6 → 5
```

4. Wait beyond 2 minutes

- It must remain:

```
Enemy interval: 5
```

- It must never become 4, 3, etc.

5. Press R

- The maze restarts.
- Three enemies remain.
- HUD immediately returns to:

```
Enemy interval: 20
```

- After another 15 seconds it becomes 18 again.

6. Verify Task 1 still works:

- All 3 enemies independently chase the player.
- Collision with any enemy causes CAUGHT!.
- EXIT still produces ESCAPED!.

7. Verify nothing from Tasks 3/4 appears:

- No power pellets.
- No score system.
- No additional mechanics.

Optional syntax check

From the repository root:

```
Bash
```

```
python -m py_compile game/game_engine.py
```

If that command produces **no output**, the file passes Python's syntax compilation check.

I am continuing from the EXISTING 11_maze_chase repository.

Tasks 1 and 2 are already implemented and working.

Now implement ONLY Task 3 from the README:

POWER PELLETT / FREEZE ENEMIES

Requirements:

- Add one visible yellow circular power pellet in a valid

Show more

For Task 3, **two files need to change**:

1. `game/entities.py` — add `self.frozen`, make `Enemy.update()` immediately return while frozen, and draw a clear frozen indicator.
2. `game/game_engine.py` — add the pellet, 300-frame freeze countdown, collection logic, and reset behavior while preserving Tasks 1 and 2.

`maze.py` and `main.py` do **not** need to change. The existing BFS implementation is also left untouched. The `Enemy.update()` structure already calls BFS, so the only change is the frozen guard before that logic.  [GitHub](#)

1. Replace `game/entities.py`

```
game/entities.py

import pygame
from game.maze import CELL

SPEED = 2

class Player:
    def __init__(self, r, c):
        self.r, self.c = r, c
        cx, cy = c * CELL + CELL // 2, r * CELL + CELL // 2
        self.rect = pygame.Rect(cx - 10, cy - 10, 20, 20)
        self.color = (60, 120, 220)
```

```

def move(self, keys, walls, rows, cols):
    dx = dy = 0

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        dx = -SPEED
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        dx = SPEED
    if keys[pygame.K_UP] or keys[pygame.K_w]:
        dy = -SPEED
    if keys[pygame.K_DOWN] or keys[pygame.K_s]:
        dy = SPEED

    nr = self.rect.move(dx, 0)
    if self._valid(nr, walls, rows, cols):
        self.rect = nr

    nr = self.rect.move(0, dy)
    if self._valid(nr, walls, rows, cols):
        self.rect = nr

def _valid(self, rect, walls, rows, cols):
    for px, py in [
        (rect.left, rect.top),
        (rect.right - 1, rect.top),
        (rect.left, rect.bottom - 1),
        (rect.right - 1, rect.bottom - 1)
    ]:
        cr, cc = py // CELL, px // CELL

        if not (0 <= cr < rows and 0 <= cc < cols):
            return False

    return True

def draw(self, screen):
    pygame.draw.ellipse(screen, self.color, self.rect)

class Enemy:
    def __init__(self, r, c):
        self.r, self.c = r, c

```

```

    cx, cy = c * CELL + CELL // 2, r * CELL + CELL // 2
    self.rect = pygame.Rect(cx - 12, cy - 12, 24, 24)

    self.color = (220, 60, 60)
    self.timer = 0
    self.move_interval = 20 # frames between cell moves

    # Task 3: Frozen state.
    self.frozen = False

def update(self, walls, player, rows, cols):
    # Task 3: A frozen enemy does not move or run BFS.
    if self.frozen:
        return

    from game.maze import bfs

    self.timer += 1

    if self.timer >= self.move_interval:
        self.timer = 0

        pr, pc = (
            player.rect.centery // CELL,
            player.rect.centerx // CELL
        )

        step = bfs(
            walls,
            (self.r, self.c),
            (pr, pc),
            rows,
            cols
        )

        if step:
            dr, dc = step
            self.r += dr
            self.c += dc

            cx = self.c * CELL + CELL // 2
            cy = self.r * CELL + CELL // 2
            self.rect.center = (cx, cy)

```

```

def draw(self, screen):
    # Normal enemy body.
    pygame.draw.rect(
        screen,
        self.color,
        self.rect,
        border_radius=5
    )

    # Eyes.
    for ex in [self.rect.x + 4, self.rect.x + 14]:
        pygame.draw.circle(
            screen,
            (255, 255, 255),
            (ex, self.rect.y + 8),
            4
        )

        pygame.draw.circle(
            screen,
            (0, 0, 0),
            (ex + 1, self.rect.y + 8),
            2
        )

    # Task 3: Clear frozen indicator.
    if self.frozen:
        pygame.draw.circle(
            screen,
            (100, 220, 255),
            self.rect.center,
            16,
            3
        )

        pygame.draw.line(
            screen,
            (100, 220, 255),
            (
                self.rect.centerx - 6,
                self.rect.centery - 6
            ),

```

```

        (
            self.rect.centerx + 6,
            self.rect.centery + 6
        ),
        3
    )

    pygame.draw.line(
        screen,
        (100, 220, 255),
        (
            self.rect.centerx + 6,
            self.rect.centery - 6
        ),
        (
            self.rect.centerx - 6,
            self.rect.centery + 6
        ),
        3
    )

```

2. Replace `game/game_engine.py`

This version preserves the **three enemies from Task 1** and the **20 → 18 → ... → 5 speed progression from Task 2**, while adding only the Task 3 functionality.

The pellet is placed in cell `(ROWS // 2, 1)`, which is a valid maze cell and does not overlap the EXIT.

```

game/game_engine.py

import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 50
FPS = 60

class GameEngine:

```

```

def __init__(self):
    pygame.init()
    self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Maze Chase")
    self.clock = pygame.time.Clock()
    self.font = pygame.font.SysFont("monospace", 22)
    self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
    self.reset()

def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)

    # Task 1: Three independent enemies.
    self.enemies = [
        Enemy(ROWS - 1, COLS - 1), # bottom-right
        Enemy(0, COLS - 1),        # top-right
        Enemy(ROWS - 1, 0),        # bottom-left
    ]

    # Task 2: All enemies start with interval 20.
    for enemy in self.enemies:
        enemy.move_interval = 20
        enemy.frozen = False

    # Task 2: Speed progression timer.
    self.speed_start_time = pygame.time.get_ticks()

    # Task 3: Power pellet.
    # Pellet is placed at a valid maze cell.
    pellet_row = ROWS // 2
    pellet_col = 1

    pellet_x = pellet_col * CELL + CELL // 2
    pellet_y = pellet_row * CELL + CELL // 2

    self.power_pellet = pygame.Rect(
        pellet_x - 8,
        pellet_y - 8,
        16,
        16
    )

```

```

self.pellet_collected = False

# Task 3: 300 frames = 5 seconds at 60 FPS.
self.freeze_frames_remaining = 0

self.exit_rect = pygame.Rect(
    (COLS // 2) * CELL + 5,
    (ROWS // 2) * CELL + 5,
    CELL - 10,
    CELL - 10
)

self.caught = False
self.won = False

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()

    return True

def update(self):
    if self.caught or self.won:
        return

    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)

    # -----
    # Task 2: Speed Up Over Time
    # -----

    elapsed_time = (
        pygame.time.get_ticks() - self.speed_start_time
    )

    speed_steps = elapsed_time // 15000

    current_interval = max(

```

```

        5,
        20 - (speed_steps * 2)
    )

    # All three enemies use the same current movement interval.
    for enemy in self.enemies:
        enemy.move_interval = current_interval

    # -----
    # Task 3: Power Pellet / Freeze Enemies
    # -----

    # Check whether the player collected the pellet.
    if (
        not self.pellet_collected
        and self.player.rect.colliderect(self.power_pellet)
    ):
        self.pellet_collected = True

        # Freeze all three enemies for exactly 300 frames.
        self.freeze_frames_remaining = 300

        for enemy in self.enemies:
            enemy.frozen = True

    # Handle the active freeze countdown.
    if self.freeze_frames_remaining > 0:
        self.freeze_frames_remaining -= 1

        if self.freeze_frames_remaining == 0:
            for enemy in self.enemies:
                enemy.frozen = False

    # -----
    # Task 1: Independently update all three enemies.
    # Enemy.update() performs the existing BFS pathfinding.
    # Frozen enemies immediately return from Enemy.update().
    # -----

    for enemy in self.enemies:
        enemy.update(
            self.walls,
            self.player,

```

```

        ROWS,
        COLS
    )

    # Check collision with every enemy.
    for enemy in self.enemies:
        if self.player.rect.colliderect(enemy.rect):
            self.caught = True
            break

    # Existing EXIT behavior.
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw(self):
    self.screen.fill((230, 220, 210))

    wc = (50, 40, 60)

    for r in range(ROWS):
        for c in range(COLS):
            x, y = c * CELL, r * CELL
            w = self.walls[r][c]

            if w[0]:
                pygame.draw.line(
                    self.screen,
                    wc,
                    (x, y),
                    (x + CELL, y),
                    3
                )

            if w[1]:
                pygame.draw.line(
                    self.screen,
                    wc,
                    (x, y + CELL),
                    (x + CELL, y + CELL),
                    3
                )

            if w[2]:

```

```

        pygame.draw.line(
            self.screen,
            wc,
            (x + CELL, y),
            (x + CELL, y + CELL),
            3
        )

    if w[3]:
        pygame.draw.line(
            self.screen,
            wc,
            (x, y),
            (x, y + CELL),
            3
        )

# Existing EXIT.
pygame.draw.rect(
    self.screen,
    (80, 200, 80),
    self.exit_rect,
    border_radius=4
)

lbl = self.font.render(
    "EXIT",
    True,
    (20, 80, 20)
)

self.screen.blit(
    lbl,
    (
        self.exit_rect.x + 2,
        self.exit_rect.y + 6
    )
)

# Task 3: Draw the power pellet while it has not been collected.
if not self.pellet_collected:
    pygame.draw.circle(
        self.screen,

```

```

        (255, 220, 0),
        self.power_pellet.center,
        8
    )

    pygame.draw.circle(
        self.screen,
        (255, 245, 120),
        self.power_pellet.center,
        4
    )

self.player.draw(self.screen)

# Task 1: Draw all three enemies.
# Frozen enemies display their frozen indicator.
for enemy in self.enemies:
    enemy.draw(self.screen)

# HUD.
hud = pygame.Rect(
    0,
    ROWS * CELL,
    WIDTH,
    50
)

pygame.draw.rect(
    self.screen,
    (30, 30, 50),
    hud
)

# Task 2: Display current enemy movement interval.
info = self.font.render(
    f"Enemy interval: {self.enemies[0].move_interval}      R=Restart"
    True,
    (200, 200, 200)
)

self.screen.blit(
    info,
    (

```

```

        8,
        ROWS * CELL + 14
    )
)

# Task 3: Display remaining freeze time when active.
if self.freeze_frames_remaining > 0:
    freeze_seconds = (
        self.freeze_frames_remaining + 59
    ) // 60

    freeze_text = self.font.render(
        f"FROZEN: {freeze_seconds}s",
        True,
        (100, 220, 255)
    )

    self.screen.blit(
        freeze_text,
        (
            WIDTH - freeze_text.get_width() - 8,
            ROWS * CELL + 14
        )
    )

if self.caught:
    self._overlay(
        "CAUGHT!",
        (220, 60, 60)
    )

if self.won:
    self._overlay(
        "ESCAPED!",
        (80, 220, 80)
    )

pygame.display.flip()

def _overlay(self, text, color):
    surf = pygame.Surface(
        (WIDTH, ROWS * CELL),
        pygame.SRCALPHA

```

```

    )

    surf.fill((0, 0, 0, 140))
    self.screen.blit(
        surf,
        (0, 0)
    )

    msg = self.big_font.render(
        text,
        True,
        color
    )

    sub = self.font.render(
        "Press R to Restart",
        True,
        (200, 200, 200)
    )

    self.screen.blit(
        msg,
        (
            WIDTH // 2 - msg.get_width() // 2,
            ROWS * CELL // 2 - 30
        )
    )

    self.screen.blit(
        sub,
        (
            WIDTH // 2 - sub.get_width() // 2,
            ROWS * CELL // 2 + 20
        )
    )

def run(self):
    running = True

    while running:
        running = self.handle_events()
        self.update()
        self.draw()

```

```
self.clock.tick(FPS)
```

```
pygame.quit()
```

Exactly what to replace

Replace **only these two files**:

```
11_maze_chase/  
└─ game/  
    └─ entities.py      ← REPLACE  
    └─ game_engine.py  ← REPLACE
```

Leave these completely unchanged:

```
main.py  
game/maze.py  
README.md
```

How Task 3 works

When the player touches the yellow pellet:

```
pellet_collected = True  
freeze_frames_remaining = 300  
enemy.frozen = True
```

Each game frame then decreases the counter:

```
300 → 299 → 298 → ... → 1 → 0
```

At `0`, all three enemies are unfrozen.

Because the game runs at 60 FPS:

```
300 frames / 60 FPS = 5 seconds
```

While frozen, `Enemy.update()` exits immediately **before the timer/BFS code**, so the enemies genuinely stop moving and do not perform BFS during the freeze. The underlying BFS implementation itself is untouched. [GitHub](#)

The pellet is also guarded by:

```
if not self.pellet_collected
```

so it can only trigger the freeze **once per game**.

Pressing **R** calls `reset()`, which recreates the pellet, sets the freeze counter back to `0`, and sets all three enemies back to `frozen = False`.

Testing checklist

1. Run:

Bash

```
python main.py
```

2. Confirm there are still **exactly 3 enemies**.
3. Confirm they independently follow the player using BFS.
4. Find the **yellow circular pellet**.
5. Touch it with the player.
6. Confirm:
 - Pellet disappears.
 - All 3 enemies immediately stop.
 - Each enemy gets the blue frozen indicator.
 - HUD shows the remaining freeze time.
7. Wait approximately **5 seconds**.
 - The enemies should become unfrozen.
 - They should resume their normal BFS movement.
8. Confirm the pellet does **not** reappear or trigger a second time.
9. Press **R**.
 - Pellet should return.
 - Enemies should no longer be frozen.
 - Freeze timer should be reset.
 - Task 2 speed progression should also restart at interval **20**.
10. Leave the game running long enough to verify Task 2 still progresses:

```
20 → 18 → 16 → 14 → 12 → 10 → 8 → 6 → 5
```

11. Confirm there is **no score system or other Task 4 functionality**.

I am continuing from the EXISTING 11_maze_chase repository.

Tasks 1, 2, and 3 are already implemented and working.

Now implement ONLY Task 4 from the README:

SURVIVAL SCORE

Requirements:

- Add `self.score` to `GameEngine`.

Show more

I checked the compatibility points first: Tasks 1–3 can remain unchanged. The three-enemy loop, Task 2's `move_interval` progression, and Task 3's pellet/freeze countdown all live correctly alongside a frame-based score. **Task 4 only requires changing** `game/game_engine.py`; `entities.py`, `maze.py`, and `main.py` **do not need changes**.

The score will increment once per active game frame, and `update()` already stops entirely once `self.caught` or `self.won` becomes true, so the score naturally stops at both endings.

File to replace

Replace **only**:

```
game/game_engine.py
```

Do **not** replace:

```
game/entities.py
game/maze.py
main.py
```

Complete `game/game_engine.py`

game/game_engine.py — Tasks 1–4

```
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy
```

```

COLS, ROWS = 13, 11
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)

        # Task 1: Three independent enemies.
        self.enemies = [
            Enemy(ROWS - 1, COLS - 1), # bottom-right
            Enemy(0, COLS - 1),        # top-right
            Enemy(ROWS - 1, 0),        # bottom-left
        ]

        # Task 2: All enemies start with interval 20.
        for enemy in self.enemies:
            enemy.move_interval = 20
            enemy.frozen = False

        # Task 2: Speed progression timer.
        self.speed_start_time = pygame.time.get_ticks()

        # Task 3: Power pellet.
        pellet_row = ROWS // 2
        pellet_col = 1

        pellet_x = pellet_col * CELL + CELL // 2
        pellet_y = pellet_row * CELL + CELL // 2

```

```

self.power_pellet = pygame.Rect(
    pellet_x - 8,
    pellet_y - 8,
    16,
    16
)

self.pellet_collected = False

# Task 3: 300 frames = 5 seconds at 60 FPS.
self.freeze_frames_remaining = 0

# Task 4: Survival score.
self.score = 0

self.exit_rect = pygame.Rect(
    (COLS // 2) * CELL + 5,
    (ROWS // 2) * CELL + 5,
    CELL - 10,
    CELL - 10
)

self.caught = False
self.won = False

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()

    return True

def update(self):
    if self.caught or self.won:
        return

    # Task 4: Add exactly 1 point for every active game frame.
    self.score += 1

    keys = pygame.key.get_pressed()

```

```

self.player.move(keys, self.walls, ROWS, COLS)

# -----
# Task 2: Speed Up Over Time
# -----

elapsed_time = (
    pygame.time.get_ticks() - self.speed_start_time
)

speed_steps = elapsed_time // 15000

current_interval = max(
    5,
    20 - (speed_steps * 2)
)

# Apply the current interval to all three enemies.
for enemy in self.enemies:
    enemy.move_interval = current_interval

# -----
# Task 3: Power Pellet / Freeze Enemies
# -----

if (
    not self.pellet_collected
    and self.player.rect.colliderect(self.power_pellet)
):
    self.pellet_collected = True

    # Freeze all three enemies for exactly 300 frames.
    self.freeze_frames_remaining = 300

    for enemy in self.enemies:
        enemy.frozen = True

# Handle the active freeze countdown.
if self.freeze_frames_remaining > 0:
    self.freeze_frames_remaining -= 1

    if self.freeze_frames_remaining == 0:
        for enemy in self.enemies:

```



```

    )

    if w[1]:
        pygame.draw.line(
            self.screen,
            wc,
            (x, y + CELL),
            (x + CELL, y + CELL),
            3
        )

    if w[2]:
        pygame.draw.line(
            self.screen,
            wc,
            (x + CELL, y),
            (x + CELL, y + CELL),
            3
        )

    if w[3]:
        pygame.draw.line(
            self.screen,
            wc,
            (x, y),
            (x, y + CELL),
            3
        )

# Existing EXIT.
pygame.draw.rect(
    self.screen,
    (80, 200, 80),
    self.exit_rect,
    border_radius=4
)

lbl = self.font.render(
    "EXIT",
    True,
    (20, 80, 20)
)

```

```
self.screen.blit(
    lbl,
    (
        self.exit_rect.x + 2,
        self.exit_rect.y + 6
    )
)

# Task 3: Draw power pellet until collected.
if not self.pellet_collected:
    pygame.draw.circle(
        self.screen,
        (255, 220, 0),
        self.power_pellet.center,
        8
    )

    pygame.draw.circle(
        self.screen,
        (255, 245, 120),
        self.power_pellet.center,
        4
    )

self.player.draw(self.screen)

# Task 1: Draw all three enemies.
for enemy in self.enemies:
    enemy.draw(self.screen)

# HUD.
hud = pygame.Rect(
    0,
    ROWS * CELL,
    WIDTH,
    50
)

pygame.draw.rect(
    self.screen,
    (30, 30, 50),
    hud
)
```

```

# Task 4: Display survival score exactly as:
# "Survived: Xs"
score_text = self.font.render(
    f"Survived: {self.score // 60}s",
    True,
    (200, 200, 200)
)

self.screen.blit(
    score_text,
    (
        8,
        ROWS * CELL + 14
    )
)

# Task 2: Display current enemy movement interval.
interval_text = self.font.render(
    f"Interval: {self.enemies[0].move_interval}",
    True,
    (200, 200, 200)
)

self.screen.blit(
    interval_text,
    (
        WIDTH - interval_text.get_width() - 8,
        ROWS * CELL + 14
    )
)

# Task 3: Display remaining freeze time when active.
if self.freeze_frames_remaining > 0:
    freeze_seconds = (
        self.freeze_frames_remaining + 59
    ) // 60

    freeze_text = self.font.render(
        f"FROZEN: {freeze_seconds}s",
        True,
        (100, 220, 255)
    )

```

```

        self.screen.blit(
            freeze_text,
            (
                WIDTH // 2 - freeze_text.get_width() // 2,
                ROWS * CELL + 14
            )
        )

    if self.caught:
        self._overlay(
            "CAUGHT!",
            (220, 60, 60)
        )

    if self.won:
        self._overlay(
            "ESCAPED!",
            (80, 220, 80)
        )

    pygame.display.flip()

def _overlay(self, text, color):
    surf = pygame.Surface(
        (WIDTH, ROWS * CELL),
        pygame.SRCALPHA
    )

    surf.fill((0, 0, 0, 140))
    self.screen.blit(
        surf,
        (0, 0)
    )

    msg = self.big_font.render(
        text,
        True,
        color
    )

    score_msg = self.font.render(
        f"Survived: {self.score // 60}s",

```

```

        True,
        (255, 255, 255)
    )

    sub = self.font.render(
        "Press R to Restart",
        True,
        (200, 200, 200)
    )

    self.screen.blit(
        msg,
        (
            WIDTH // 2 - msg.get_width() // 2,
            ROWS * CELL // 2 - 55
        )
    )

    self.screen.blit(
        score_msg,
        (
            WIDTH // 2 - score_msg.get_width() // 2,
            ROWS * CELL // 2
        )
    )

    self.screen.blit(
        sub,
        (
            WIDTH // 2 - sub.get_width() // 2,
            ROWS * CELL // 2 + 35
        )
    )

def run(self):
    running = True

    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)

```

```
pygame.quit()
```

Why this preserves Tasks 1–3

- **Task 1:** `self.enemies` remains a list of exactly 3 enemies, and all three still run through `enemy.update()`.
- **Task 2:** `pygame.time.get_ticks()` still controls the progression:
`20 → 18 → 16 → 14 → 12 → 10 → 8 → 6 → 5`.
- **Task 3:** The yellow pellet, `300`-frame freeze, `enemy.frozen`, and frozen indicators remain unchanged.
- **Task 4:** `self.score` is reset to `0` in `reset()` and increments once at the beginning of every active `update()` frame.
- Once `self.caught` or `self.won` becomes `True`, the first line of `update()` returns, so the **score cannot increase afterward**.
- Pressing **R** calls `reset()`, resetting both the score and all previous Task 1–3 state.

The existing project structure confirms that the game loop and enemy behavior are separated between `game_engine.py` and `entities.py`, so no change to the maze or entry point is necessary.  [GitHub](#) +1

Testing checklist

Run:

Bash

```
python main.py
```

Then verify:

1. HUD initially shows:

```
Survived: 0s
```

2. After roughly 1 second:

```
Survived: 1s
```

3. Continue playing and confirm the number increases once per 60 frames.
4. Get caught by an enemy:
 - Game stops.
 - `Survived: Xs` stops increasing.

- The caught overlay displays the **final** `Survived: Xs`.

5. Alternatively, reach the EXIT:

- Game stops.
- Score stops increasing.
- `ESCAPED!` overlay displays the final survival score.

6. Verify Task 2 still progresses:

20 → 18 → 16 → 14 → 12 → 10 → 8 → 6 → 5

7. Collect the yellow pellet:

- Pellet disappears.
- All 3 enemies freeze for 300 frames / 5 seconds.
- Frozen indicators appear.
- Enemies resume BFS movement afterward.

8. Press **R**:

- Score returns to `Survived: 0s`.
- Speed interval returns to `20`.
- Pellet returns.
- Freeze state resets.
- All 3 enemies are active again.

9. Confirm **no additional Task 4 mechanics beyond the survival score** have been introduced.